Study Material 4

1. Please answer to the following questions <u>in your own words</u>.

(a) (Exercise 6.5) An E-R diagram can be viewed as a graph. What do the following mean in terms of the structure of an enterprise schema?
- The graph is disconnected.
- The graph has a cycle.

a. If a pair of entity sets are connected by a path in an E-R diagram, the entity sets are related, though perhaps indirectly. A disconnected graph implies that there are pairs of entity sets that are unrelated to each other. In an enterprise, we can say that the two parts of the enterprise are completely independent of each other. If we split the graph into connected components, we have, in effect, a separate database corresponding to each independent part of the enterprise.

b. As indicated in the answer to the previous part, a path in the graph between a pair of entity sets indicates a (possibly indirect) relationship between the two entity sets. If there is a cycle in the graph, then every pair of entity sets on the cycle are related to each other in at least two distinct ways. If the E-R diagram is acyclic, then there is a unique path between every pair of entity sets and thus a unique relationship between every pair of entity sets.

(b) (Exercise 6.17) Explain the difference between a weak and a strong entity set.

A strong entity set has a primary key. All entities in the set are distinguishable by that key. A weak entity set has no primary key unless attributes of the strong entity set on which it depends are included. Entities in a weak entity set are partitioned according to their relationship with entities in a strong entity set.

Entities within each partition are distinguishable by a discriminator, which is a set of attributes.

(c) (Exercise 6.19) We can convert any weak entity set to a strong entity set by simply adding appropriate attributes. Why, then, do we have weak entity sets?

- To avoid the data duplication and consequent possible inconsistencies caused by duplicating the key of the strong entity set.
- Weak entities reflect the logical structure of an entity being dependent on another entity.
- Weak entities can be deleted automatically when their strong entity is deleted.

(d) (Exercise 7.10) Our discussion of lossless decomposition implicitly assumed that attributes on the left-hand side of a functional dependency cannot take on null values. What could go wrong on decomposition, if this property is violated?

The natural join operator is defined in terms of the Cartesian product and the selection operator. The selection operator gives *unknown* for any query on a null value. Thus, the natural join excludes all tuples with null values on the common attributes from the final result. Thus, the decomposition would be lossy (in a manner different from the usual case of lossy decomposition), if null values occur in the left-hand side of the functional dependency used to decompose the relation. (Null values in attributes that occur only in the right-hand side of the functional dependency do not cause any problems.)

(e) (Exercise 7.21) *Repetition of information* and *inability to represent information* can be defined as next:
- *Repetition of information*: a condition in a relational database where the values of one attribute are determined by the values of another attribute in the same relation, and both values are repeated throughout the relation.
- *Inability to represent information*: a condition where there is a relation- ship that exists among only a proper subset of the attributes in a relation.
Explain why each of these properties may indicate a bad relational-database design.

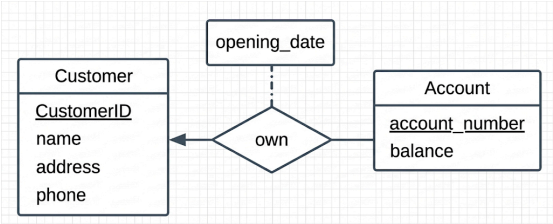
Inability to represent information can also occur because of loss of information that results from the decomposition of one relation into two relations that cannot be combined to re-create the original relation. Such a lossy decomposition may happen implicitly, even without explicitly carrying out decomposition, if the initial relational schema itself corresponds to the decomposition.

(f) (Exercise 7.22) Why are certain functional dependencies called *trivial* functional dependencies?

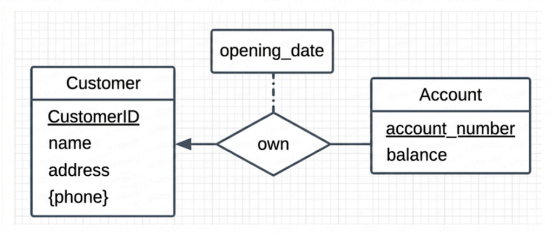
Certain functional dependencies are called trivial functional dependencies because they are satisfied by all relations

2. Draw the E-R diagrams for the following databases. Be sure to indicate the cardinalities of the relationship.

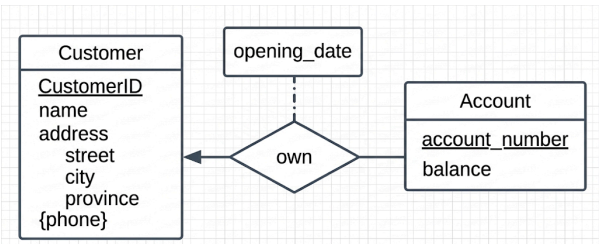
(a) Design a database for a bank, including information about *customers*, their *accounts*, and the *own* relationship between them. Information about a *customer* includes their *name, address, phone*, and *customer ID*. An *Account* has an *account number* and *balance*. Also, the *own* relationship keeps *opening date* of each *account*. Note that:
- A *customer* can *own* multiple accounts.
- An *account* is *own*ed by only one customer.
- *Customer ID* and *Account number* are unique to each *customer* and *account*, respectively.



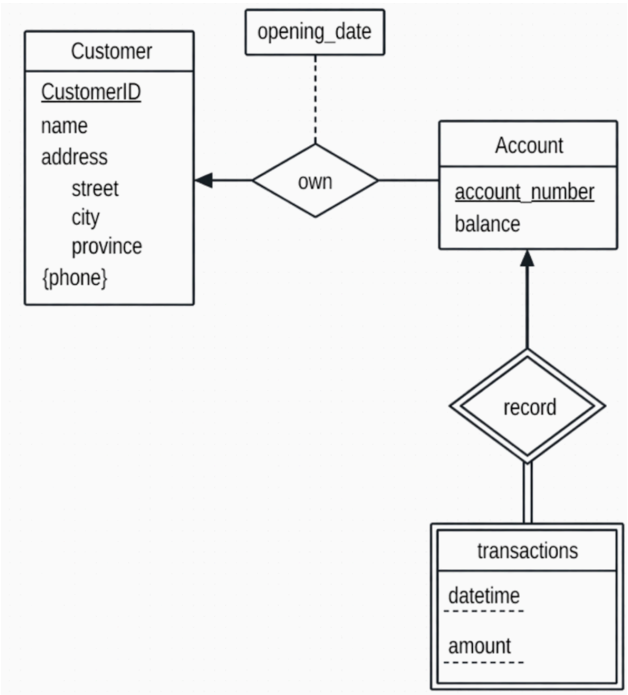
(b) Modify your original diagram of Problem 2(a) such that a *customer* can have multiple *phone* numbers.



(c) Change your diagram of Problem 2(b) such that a *customer* has an *address* represented by composite attributes (which are *street-city-province* triplets). Note that multiple customers may live at a single address.



(d) Add a weak entity set of *transaction*s next to *account* (connected via a relationship *record*). This entity set contains *transaction datetime* and *amount* as its attributes.



(e) Convert the following E-R diagram into SQL DDL (CREATE TABLE statements).
- Consider the cardinalities of the relationship.
- Recall that we do not allow attributes to have non-primitive data types.

Customer:Account = 1:many
Customer:Address = many:1
Customer:Phone = 1:many
Account:Transaction = 1:many

Sample schema design:
Customer(name, address_ID, customer_ID)
Phone(customer_ID, phone_number)
Address(address_ID, street, city, province)
Account(customer_ID, account_number, balance, opening_date)
Transactions(account_number, datetime, amount)
Sample SQL DDL:

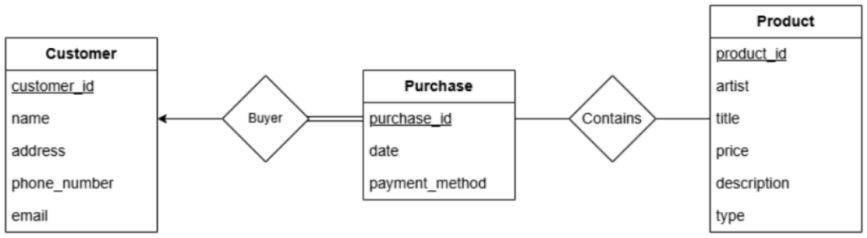
```
CREATE TABLE Address (
   address_ID INT PRIMARY KEY,
   street VARCHAR(100),
   city VARCHAR(50),
   province VARCHAR(50)
);

CREATE TABLE Customer (
   customer_ID INT PRIMARY KEY,
   name VARCHAR(50),
   address_ID INT UNIQUE,
   FOREIGN KEY (address_ID) REFERENCES Address(address_ID)
);

CREATE TABLE Phone (
   customer_ID INT,
   phone_number VARCHAR(20),
   PRIMARY KEY (customer_ID, phone_number),
   FOREIGN KEY (customer_ID) REFERENCES Customer(customer_ID)
);

CREATE TABLE Account (
   account_number INT PRIMARY KEY,
   customer_ID INT,
   balance DECIMAL(15,2),
   opening_date DATE,
   FOREIGN KEY (customer_ID) REFERENCES Customer(customer_ID)
);

CREATE TABLE Transactions (
   account_number INT,
   datetime TIMESTAMP,
   amount INT,
   PRIMARY KEY (account_number, datetime, amount),
   FOREIGN KEY (account_number) REFERENCES Account(account_number)
);
```

Alternative sample schema design (there could be other possible solutions):
Customer(name, customer_ID)
Phone(customer_ID, phone_number)
Address(customer_ID, street, city, province)
Account(customer_ID, account_number, balance, opening_date)
Transactions(account_number, datetime, amount)

(f) Draw an ER diagram for Record Shop Database

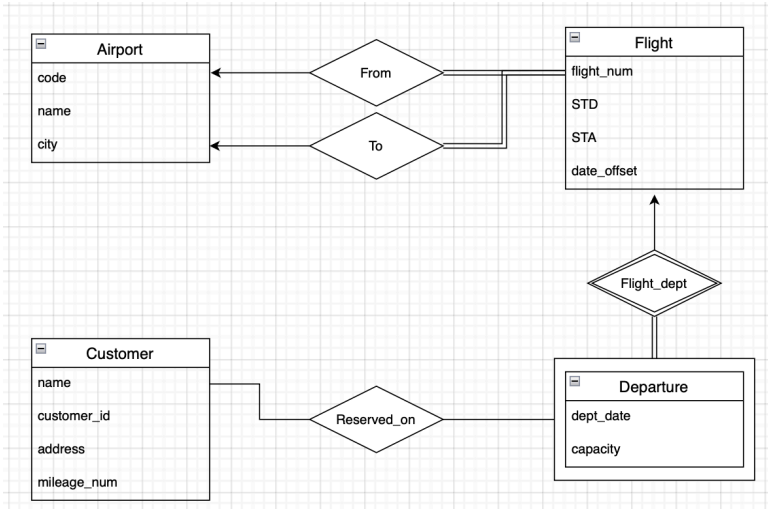
Record Shop
Entity Set
Customer(customer_id, name, address, phone_number, email)
Product(product_id, artist, title, price, description, type)
Purchase(purchase_id, date, payment_method)
Relationship Set
Contains(between product and purchase) (many to many)
Buyer(between customer and purchase) (1 to many)

Sample solution:



(g) Draw an ER diagram for Flight Database
Flight Database
Entity Set
Airport(code, name, city)
Flight(flight_num, STD, STA, date_offset) * *Note: Flight Entity represents the flight route*
Departure(dept_date, capacity) * *Note: Departure Entity represents the actual schedule*
Customer(customer_id, name, address, mileage_num)
Relationship Set
To(flight, airport)
From(flight, airport)
Flight_dept(flight, departure)
Reserved_On(customer, departure)

Sample solution:



3. Normalization

(a) Is every relation in 3NF also in BCNF? If yes, explain why. If no, given a counter example.

No

3NF: R contains only columns that are non-transitively dependent on the primary key.

This DOES NOT automatically satisfy BCNF:

A relation schema R is in Boyce-Codd Normal Form (BCNF) if whenever a nontrivial FD X → A holds in R, then X is a superkey of R

For the following (**student_id**, **subject**, instructor) satisfies 3NF but BCNF, because subject → instructor holds whereas subject is not a superkey.

student_id	subject	instructor
21800999	C++	Dr. Cpp
21800999	Java	Dr. Java
21800998	C++	Dr. C
21800997	Python	Dr. Python
21800996	C++	Dr. Cpp

To satisfy BCNF, one may decompose the above as:

(**student_id**, **section_id**)

(**section_id**, subject, instructor)

student_id	section_id
21800999	101
21800999	103
21800998	102
21800997	104
21800996	101

section_id	subject	instructor
101	C++	Dr. Cpp
103	Java	Dr. Java
102	C++	Dr. C
104	Python	Dr. Python

(b) Is every relation in 4NF also in BCNF? If yes, explain why. If no, given a counter example.

Yes (being in BCNF is a precondition to satisfy 4NF)

(c) The following relation violates {1NF, 2NF, 3NF, 4NF, BCNF}?

employee_id	**name**	**branch**	**project**
1001	Brown	Seoul	Mars, Saturn
1004	Green	Seoul	Saturn
1005	Hopkins	Pohang	Jupiter, Saturn
2001	Kim	Seoul	Mars
3002	Walker	Seoul	Venus
3004	Kim	Pohang	Venus
3005	Clermont	Pohang	Jupiter, Saturn

1NF (values must be atomic)

(d) The following relation violates {1NF, 2NF, 3NF, 4NF, BCNF}?

employee_id	**name**	**branch**	**branch_address**
1001	Brown	Seoul	Garosu-gil 233

1004	Green	Seoul	Garosu-gil 233
1005	Hopkins	Pohang	Handong-ro 501
2001	Kim	Seoul	Garosu-gil 233
3002	Walker	Seoul	Garosu-gil 233
3004	Kim	Pohang	Handong-ro 501
3005	Clermont	Pohang	Handong-ro 501

2NF (contains partial dependency (branch → branch_address))

(e) The following relation violates {1NF, 2NF, 3NF, 4NF, BCNF}?

employee_id	**name**	**branch**	**dept_id**	**dept_name**
1001	Brown	Seoul	202	Sales
1004	Green	Seoul	201	Operation
1005	Hopkins	Pohang	303	Software development
2001	Kim	Seoul	303	Software development
3002	Walker	Seoul	303	Software development
3004	Kim	Pohang	308	User experience
3005	Clermont	Pohang	201	Operation

BCNF dept_id → dept_name, where dept_id is a non-PK